بخش هشتم: ارزیابی الگوریتم‌ها (Algorithm Evaluation)

(بر اساس فایل pdf.8 - پایان فصل ۵)

سوال اصلی: چگونه بهترین الگوریتم زمان‌بندی را برای یک سیستم خاص انتخاب کنیم؟ برای پاسخ به این سوال، ابتدا باید **معیارها (Criteria)** را مشخص کنیم (مثلاً: "حداکثر زمان پاسخ باید زیر ۳۰۰ میلی‌ثانیه باشد"). سپس از یکی از روش‌های زیر برای ارزیابی استفاده می‌کنیم:

۱. مدل‌سازی معین (Deterministic Modeling)

این روش نوعی **ارزیابی تحلیلی** است.

> **روش کار:** یک "بارکاری از پیش تعریف شده" (Workload) شامل مجموعه‌ای از پردازه‌ها با زمان‌های ورود و انفجار مشخص را در نظر می‌گیریم. سپس عملکرد هر الگوریتم را دقیقاً برای همان داده‌ها محاسبه می‌کنیم.

مثال حل شده: فرض کنید ۵ پردازه با طول انفجار زیر در لحظه ۰ وارد می‌شوند:

- P1: 10
- P2: 29
- P3: 3
- P4: 7
- P5: 12

مقایسه الگوریتم‌ها:

1. **FCFS:** میانگین زمان انتظار = **۲۸** میلی‌ثانیه.
2. **SJF (غیرپس‌گرفتنی):** میانگین زمان انتظار = **۱۳** میلی‌ثانیه.
3. **RR (کوانتوم ۱۰):** میانگین زمان انتظار = **۲۳** میلی‌ثانیه.

- **نتیجه:** در این مثال خاص، SJF بهترین است (کمتر از نصف FCFS انتظار دارد).
- **مزایا:** ساده، سریع و دقیق (برای همان داده‌های خاص).
- **معایب:** نتایج قابل تعمیم نیستند (ممکن است با تغییر اعداد، نتیجه عوض شود).

۲. مدل‌های صف‌بندی (Queueing Models)

در سیستم‌های واقعی، بار کاری ثابت نیست. در این روش از **توزیع‌های احتمال** (ریاضیات و آمار) استفاده می‌کنیم.

- توزیع زمان ورود (Arrival Distribution).
- توزیع زمان سرویس (Service/Burst Distribution).

فرمول لیتل (Little's Formula): ⋆ *بسیار مهم* رابطه‌ای قدرتمند که برای هر سیستم صفی در حالت پایدار صادق است:

n = lambda * W

- **n:** طول متوسط صف (تعداد پردازه‌های منتظر).
- **lambda (لاندا):** نرخ متوسط ورود پردازه‌ها به صف (مثلاً ۳ پردازه در ثانیه).
- **W:** میانگین زمان انتظار در صف.

کاربرد: اگر دو متغیر را داشته باشیم، سومی را پیدا می‌کنیم. (مثلاً اگر بدانیم ۷ پردازه در ثانیه می‌آیند و ۱۴ پردازه در صف هستند، پس میانگین انتظار ۲ ثانیه است).

معایب: محدودیت‌های ریاضی دارد و نیاز به فرضیات ساده‌ساز دارد که ممکن است همیشه دقیق نباشد.

۳. شبیه‌سازی (Simulation)

برای دقت بیشتر، مدل کامپیوتری سیستم را برنامه‌نویسی می‌کنیم.

داده‌های ورودی:

1. **تولید تصادفی:** با استفاده از توزیع‌های ریاضی. (مشکل: ترتیب رخدادها را نشان نمی‌دهد).
2. **فایل‌های ردیاب (Trace Files):** ⋆ *بهترین روش.*
   - یک سیستم واقعی را مانیتور می‌کنیم و تمام رخدادها (زمان ورود، زمان CPU، زمان I/O) را ضبط می‌کنیم.
   - سپس این فایل ضبط شده را به شبیه‌ساز می‌دهیم.
   - **مزیت:** مقایسه الگوریتم‌ها روی داده‌های کاملاً واقعی و یکسان.

۴. پیاده‌سازی (Implementation)

دقیق‌ترین روش این است که الگوریتم را کدنویسی کنیم و در هسته سیستم‌عامل قرار دهیم تا در دنیای واقعی کار کند.

معایب و چالش‌ها:

1. **هزینه بالا:** کدنویسی و دیباگ کرنل سخت است.
2. **تغییر رفتار کاربران:** کاربران ممکن است خودشان را با الگوریتم جدید وفق دهند و سیستم را گول بزنند!
    - **مثال واقعی:** سیستمی طراحی شد که به پردازه‌های تعاملی (که خروجی ترمینال داشتند) اولویت بالا می‌داد. یک برنامه‌نویس، برنامه‌اش را طوری نوشت که هر ۱ ثانیه یک کاراکتر بی‌معنی روی صفحه بنویسد تا سیستم‌عامل فکر کند این برنامه تعاملی است و اولویتش را بالا ببرد!

۵-۹ خلاصه فصل ۵

- **هدف:** ماکسیمم کردن بهره‌وری CPU با سوییچ کردن بین پردازه‌ها.
- **چرخه:** پردازه‌ها بین دو حالت **انفجار CPU** و **انفجار I/O** در گردش‌اند.
- **الگوریتم‌ها:**
    - **FCFS:** ساده اما با مشکل اثر بدرقه.
    - **SJF:** بهینه (کمترین انتظار)، اما نیاز به پیش‌بینی آینده دارد.
    - **RR:** مناسب برای سیستم‌های اشتراک زمانی (Time-Sharing).
    - **Priority:** مشکل گرسنگی دارد (راه حل: Aging).
    - **MLFQ:** صف‌های چندسطحی با بازخورد (استاندارد سیستم‌های مدرن).
- **سیستم‌های واقعی:**
    - **لینوکس:** استفاده از CFS (کاملاً منصف) و زمان‌بندی بلادرنگ.
    - **ویندوز:** استفاده از اولویت‌های ۳۲ سطحی و تغییر پویا (Boosting).
- **ارزیابی:** مدل‌سازی معین، صف‌بندی، شبیه‌سازی و پیاده‌سازی.

پایان فصل ۵ - سیستم‌عامل